PROJETO G2 / MARCO 3 / ISSUE #18

Uma nova tarefa, do clique até a lista.

Caderno de estudo · SvelteKit e TypeScript · 21/10/2026

01 / O que estamos construindo?

A issue #18 pede uma página em `/tarefas/nova`. Nela, a pessoa preenche os dados, salva a tarefa na API e volta para a lista de tarefas.

Também há um atalho: se a pessoa começar dentro de um projeto, esse projeto deve aparecer selecionado no formulário.

1. A pessoa abre **Nova tarefa**.
2. A página busca os projetos e mostra o formulário.
3. A pessoa preenche e envia os dados.
4. A API grava a tarefa e gera o ID.
5. O site abre a lista de tarefas novamente.

Limite desta entrega: o formulário fica nesta página. Transformá-lo em um componente reutilizável para criar e editar pertence à #17. A função de criação dá o suporte necessário à #18; isso não significa concluir toda a API de escrita da #16 nem toda a validação da #20.

02 / Onde encontrar o código

No VS Code, pressione **CmdP**, digite um dos caminhos abaixo e pressione Enter. O nome `+page.svelte` aparece várias vezes porque cada pasta representa uma página diferente.

Arquivo, a partir da raiz do projeto	Responsabilidade
`src/routes/tarefas/nova/+page.svelte`	Mostra o cadastro, guarda os valores preenchidos e coordena o envio.
`src/lib/servicos/api.ts`	Conversa com a API. A função `criarTarefa` faz a requisição de criação.
`src/tipos.ts`	Define os formatos `Tarefa` e `NovaTarefa`, usados pelo TypeScript.
`src/routes/+page.svelte`	É a lista, no endereço `/`. Tem a entrada para criar uma tarefa.
`src/routes/projetos/[id]/+page.svelte`	É o detalhe de um projeto. Seu link leva o ID do projeto até o cadastro.

Um arquivo Svelte costuma juntar três partes:

- `<script lang="ts">`: TypeScript, com dados e funções.

- Marcação HTML/Svelte: os campos, textos e botões que aparecem.

- `<style>`: CSS, que organiza a aparência da página.

Os exemplos deste guia são recortes para estudar os conceitos; os arquivos do projeto são a referência para o código completo.

03 / Como o endereço encontra a página?

```
src/routes/tarefas/nova/+page.svelte
                    v
               /tarefas/nova

src/routes/tarefas/[id]/+page.svelte
                    v
           /tarefas/2, /tarefas/abc...
```

`nova` é uma parte fixa do endereço. `[id]` é uma parte variável. O SvelteKit escolhe a rota fixa antes da variável; por isso, abrir `/tarefas/nova` deve mostrar o cadastro, sem tentar buscar uma tarefa cujo ID seja “nova”.

E o projeto já selecionado?

O link do projeto leva uma informação extra depois do ponto de interrogação:

```
/tarefas/nova?projetoId=3
```

`/tarefas/nova` escolhe a página. `projetoId=3` sugere qual projeto selecionar. Isso é um **parâmetro de consulta**, também chamado de *query parameter*.

```
import { page } from '$app/state';

page.url.searchParams.get('projetoId'); // "3"
```

`page` vem do SvelteKit e informa a URL atual. Ele não é o roteador. Também não confundimos a consulta acima com `page.params.id`, que lê a parte `[id]` de uma rota dinâmica.

Antes de selecionar, o código confere se esse ID existe nos projetos que vieram da API. Se alguém escrever um ID inválido na URL, o formulário deve orientar a pessoa a escolher um projeto válido.

Por que usar `$derived` e `$effect` aqui?

O SvelteKit pode manter a mesma página aberta quando só a consulta muda. Por isso, observar a sugestão da URL permite atualizar o projeto ao ir de `?projetoId=1` para `?projetoId=2`. O efeito observa a sugestão e os projetos carregados; ele não deve observar o campo que a pessoa edita, para não desfazer a escolha manual dela.

04 / Como o formulário guarda o que eu digito?

Os dados enviados são: `projetoId`, `titulo`, `descricao`, `status`, `prioridade`, `responsavel` e `prazo`. Os nomes precisam respeitar o contrato que a API usa.

`NovaTarefa`: o formato dos dados

```
export type NovaTarefa = Omit<Tarefa, 'id'>;
```

`Omit` significa “usar os campos de `Tarefa`, retirando `id`”. O usuário não inventa o ID. O `json-server` o gera quando grava a tarefa.

O TypeScript ajuda a detectar nomes e tipos errados enquanto programamos. Ele não substitui as verificações do formulário nem a validação de um servidor real.

`$state`: um valor que atualiza a tela

```
let titulo = $state('');
```

Esse recorte começa com um título vazio. Quando o estado muda, o Svelte atualiza a parte da tela que depende dele. Na página completa, os campos podem estar agrupados em um objeto.

`bind:value`: ligar o campo ao estado

```
<label for="titulo">Título</label>
<input id="titulo" bind:value={titulo} required />
```

Ao digitar no campo, `titulo` recebe o valor. Se o código mudar `titulo`, o campo também muda. O `label` dá nome ao campo; o `for` aponta para seu `id`. `required` pede ao navegador que não envie esse campo vazio.

`onMount`: buscar as opções depois de abrir

`listarProjetos()` fornece as opções do seletor. `onMount` executa a busca quando a página é montada no navegador. Enquanto a resposta não chega, mostramos carregamento; se falhar, mostramos uma mensagem e permitimos tentar de novo.

Duas ideias diferentes: `onMount` inicia essa busca. O efeito acompanha a sugestão de projeto da URL. Um não substitui automaticamente o outro.

05 / O que acontece ao clicar em Salvar?

Um formulário trata seu envio com uma função. Ela impede o envio tradicional do navegador, prepara os dados e aguarda a API.

Operação	O que ela faz no projeto
`GET /projetos`	Lê os projetos para o seletor.
`POST /tarefas`	Envia os dados para criar uma tarefa.
`goto('/')`	Abre a página da lista depois da confirmação de criação.

Recorte do formato de uma requisição de criação:

```
fetch('http://localhost:3000/tarefas', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(dados)
});
```

- `method: 'POST'` informa que queremos criar um registro.

- `Content-Type` informa que o corpo usa JSON.

- `JSON.stringify(dados)` transforma o objeto TypeScript/JavaScript em texto JSON para enviar.

- `criarTarefa(dados)` reúne essa conversa com a API em uma função que a página chama.

Por que `async` e `await`?

```
const tarefaCriada = await criarTarefa(dados);
await goto('/');
```

A API leva algum tempo para responder. `await` espera o resultado dentro de uma função `async`. Só depois de criar a tarefa a navegação é iniciada.

A lista fica em `/`, e não em `/tarefas`. Ao voltar para ela, a busca da lista carrega os registros, incluindo a tarefa nova.

E se der erro?

`try` tenta a operação e `catch` trata uma falha. Se a gravação falhar, os campos continuam preenchidos para a pessoa tentar novamente. Durante o envio, o botão fica desabilitado para evitar pedidos repetidos.

Se a API já confirmou a criação, uma falha ao navegar não deve habilitar uma segunda gravação da mesma tarefa. A página reconhece o sucesso e permite voltar à lista. O `onDestroy` também marca quando você saiu desta tela, para uma resposta atrasada não forçar a navegação depois disso.

A página também verifica se ainda está aberta antes de navegar: se você saiu durante o envio, a resposta não deve mudar sua nova página.

06 / Como abrir pelo terminal

Esta entrega usa uma cópia de trabalho separada. A raiz dessa cópia é:

```
/Users/andreybuenoisoton/Documents/Codex/2026-10-14/create-a-new-site-with-sites-4/outputs/marco3-issue18
```

Abra essa pasta no VS Code. No terminal integrado, use `pwd` para confirmar onde você está. Todos os comandos abaixo partem da raiz do projeto, onde está `package.json`.

1. Instalar as dependências, quando necessário

```
npm ci
```

Instala as versões registradas no `package-lock.json`. É necessário em uma cópia recém-baixada ou quando as dependências precisam ser reinstaladas. O projeto pede Node.js 24 ou superior; `node --version` mostra sua versão.

2. Abrir a API em uma aba do terminal

```
npm run api
```

Inicia o `json-server` usando o `db.json` dessa pasta. O endereço esperado pela aplicação é `http://localhost:3000`. Esse terminal continua ocupado atendendo à API.

3. Abrir o site em outra aba do terminal

```
npm run dev
```

Inicia o servidor de desenvolvimento. Abra o endereço **Local** que ele imprimir, normalmente `http://localhost:5173`, e acrescente `/tarefas/nova`.

Não copie o prompt. Um trecho como `andrey@Mac projeto-g2-frontend %` apenas mostra o usuário e a pasta atual. Você digita somente o comando depois do `%`.

Com um servidor rodando, aquela aba não está pronta para comandos Git. Abra outra aba. Para encerrar um servidor que você iniciou, use **Ctrl+C** na aba dele.

O site e a API são dois programas. A porta **5173** mostra a interface; a porta **3000** responde com os dados. Abrir a API diretamente no navegador pode mostrar JSON, o que é esperado.

Conferir os tipos e preparar a aplicação

```
npm run check
npm run build
```

O primeiro procura problemas de tipos e de Svelte. O segundo gera a versão de produção. Eles ajudam a conferir o código, mas não substituem experimentar o formulário no navegador.

07 / O que você deve experimentar

Use esta lista como roteiro de estudo. As caixas são suas marcações locais e não representam um relatório automático de testes.

- Abrir `/tarefas/nova` e identificar a tela de cadastro.

- Abrir um projeto e usar seu link de nova tarefa. Conferir qual opção já vem selecionada.

- Trocar o projeto manualmente e perceber que sua escolha é mantida.

- Preencher todos os campos e salvar uma tarefa de teste. Conferir se ela aparece na lista.

- Observar o botão durante o envio e explicar por que ele fica desabilitado.

- Abrir um detalhe como `/tarefas/2` e perceber que continua sendo outra página.

O que foi verificado nesta entrega

O Svelte encontrou zero erros e avisos, a compilação passou e três testes automatizados da API passaram. No navegador, conferimos cadastro, projeto pré-selecionado, troca manual, campos obrigatórios, clique duplo e recuperação após falha de conexão, usando uma cópia do banco.

Criar uma tarefa grava um registro no banco da API em uso. Para estudar sem alterar os dados da equipe, faça os testes em uma cópia do banco.

08 / Agora explique com suas palavras

Tente responder antes de abrir a resposta. Se travar, volte ao trecho do guia e procure a ligação entre a URL, o campo e a API.

1. Por que /tarefas/nova não abre o detalhe de uma tarefa chamada “nova”?

Porque existe uma rota fixa em `tarefas/nova/+page.svelte`. O SvelteKit dá prioridade à rota fixa sobre `tarefas/[id]/+page.svelte`.

2. De onde vem o projeto que aparece selecionado?

Do parâmetro `projetoId` na consulta da URL, enviado pelo link da página do projeto. O código lê essa sugestão e confere se ela corresponde a um projeto existente nas opções da API.

3. Qual é a diferença entre GET e POST nesta tela?

`GET` lê os projetos. `POST` envia os dados para criar a tarefa. A requisição de criação leva um corpo JSON.

4. Por que não enviamos um id junto com os campos?

O ID é gerado pela API. O tipo `NovaTarefa` usa os campos de `Tarefa` sem `id`, deixando isso claro enquanto programamos.

5. O que aconteceria se voltássemos para a lista antes de esperar a API?

A pessoa poderia ver a lista antes da gravação terminar ou sair do formulário sem perceber uma falha. A ordem correta é confirmar a criação e depois navegar.

6. O que bind:value faz quando você digita um título?

Atualiza o estado ligado ao campo. A ligação também funciona no outro sentido: quando o código altera esse estado, o valor mostrado no campo acompanha a mudança.

Para consultar quando precisar

- SvelteKit: páginas e rotas
- SvelteKit: prioridade entre rotas
- SvelteKit: informações da página atual
- SvelteKit: navegar com goto
- SvelteKit: preservação da página ao navegar
- Svelte: estado com $state
- Svelte: ligação com bind